

SALUS SECURITY

SEP 2026



CODE SECURITY ASSESSMENT

BOUNCE FINANCE

Overview

Project Summary

- Name: Bounce - Swap
- Platform: EVM-compatible chains
- Language: Solidity
- Audit Range: See [Appendix - 1](#)

Project Dashboard

Application Summary

Name	Bounce - Swap
Version	v2
Type	Solidity
Dates	Sep 18 2026
Logs	Sep 09 2026; Sep 18 2026

Vulnerability Summary

Total High-Severity issues	0
Total Medium-Severity issues	1
Total Low-Severity issues	4
Total informational issues	2
Total	7

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction	4
1.1 About SALUS	4
1.2 Audit Breakdown	4
1.3 Disclaimer	4
Findings	5
2.1 Summary of Findings	5
2.2 Notable Findings	6
1. Centralization risk	6
2. Fee-on-Transfer Payment Tokens Cause Underpayment	7
3. Global Fee Changes Retroactively Affect Existing Pools	8
4. Pool-Creation Signatures Do Not Bind Pool Terms	9
5. Small Swaps May Pay Tokens but Receive Zero Output	10
2.3 Informational Findings	11
6. Critical Configuration Changes Lack Events	11
7. Unreachable Legacy Claim Branch	12
Appendix	13
Appendix 1 - Files in Scope	13

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (<https://t.me/salusec>), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Centralization risk	Medium	Centralization	Acknowledged
2	Fee-on-Transfer Payment Tokens Cause Underpayment	Low	Business Logic	Acknowledged
3	Global Fee Changes Retroactively Affect Existing Pools	Low	Business Logic	Acknowledged
4	Pool-Creation Signatures Do Not Bind Pool Terms	Low	Business Logic	Acknowledged
5	Small Swaps May Pay Tokens but Receive Zero Output	Low	Business Logic	Acknowledged
6	Critical Configuration Changes Lack Events	Informational	Code Quality	Acknowledged
7	Unreachable Legacy Claim Branch	Informational	Business Logic	Acknowledged

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Centralization risk	
Severity: Medium	Category: Centralization
Target: - contracts/BounceFixedSwap.sol	

Description

In the BounceBase contract, there exists one privileged role, `owner`. This role has the authority to execute some key functions such as `setStakeContract`, `setTxFeeRatio`, etc.

If these roles' private keys are compromised, an attacker could trigger these functions to harm the whole protocol.

contracts/BounceFixedSwap.sol:L187-L193

```
function setTxFeeRatio(uint256 _txFeeRatio) external onlyOwner {
    _setTxFeeRatio(_txFeeRatio);
}

function setStakeContract(address _stakeContract) external onlyOwner {
    _setStakeContract(_stakeContract);
}
```

Recommendation

We recommend transferring ownership to a multi-signature account protected by a timelock. This ensures that no single party can unilaterally redirect treasury flows and gives users advance notice of treasury changes.

Status

This issue has been acknowledged by the team.

2. Fee-on-Transfer Payment Tokens Cause Underpayment

Severity: Low

Category: Business Logic

Target:

- contracts/BounceFixedSwap.sol

Description

The function `\_swap` records and prices the purchase using the requested `amount1`, but does not verify the amount of ERC-20 `token1` actually received. Fee-on-transfer tokens can deliver less than the requested amount.

Buyers can receive the full `token0` allocation while the pool receives less `token1`. Creator/protocol settlement may be underfunded or revert.

contracts/BounceFixedSwap.sol:L237-L243

```
function _swap(uint256 index, uint256 amount1) private {  
    // transfer amount of token1 to this contract  
    if (pool.token1 == address(0)) {  
        require(msg.value == amount1, "invalid amount of ETH");  
    } else {  
        IERC20Upgradeable(pool.token1).safeTransferFrom(msg.sender, address(this),  
amount1);  
    }  
}
```

Recommendation

Measure the `token1` balance before and after transfer, and require the received amount to equal `amount1`; reuse `transferAndCheck` for ERC-20 payments.

Status

This issue has been acknowledged by the team.

3. Global Fee Changes Retroactively Affect Existing Pools

Severity: Low

Category: Business Logic

Target:

- contracts/BounceFixedSwap.sol

Description

The function `creatorClaim` calculates fees using the mutable global `txFeeRatio`. The owner can update this value after a pool is created or has already received purchases.

A fee increase retroactively reduces the creator's proceeds from existing pools. At 100%, all settled proceeds are directed to the fee recipient.

contracts/BounceFixedSwap.sol:L281-L282

```
function creatorClaim(uint256 index) external nonReentrant isPoolExist(index) {  
    ...  
    uint256 txFee = amountSwap1P[index].mul(txFeeRatio).div(TX_FEE_DENOMINATOR);  
    uint256 _amount1 = amountSwap1P[index].sub(txFee);  
    ...  
}
```

Recommendation

Snapshot `txFeeRatio` per pool at creation and use the stored value during `creatorClaim`.

Status

This issue has been acknowledged by the team.

4. Pool-Creation Signatures Do Not Bind Pool Terms

Severity: Low

Category: Business Logic

Target:

- contracts/BounceFixedSwap.sol

Description

The creator signature covers only ``id`` and ``PoolType.FixedSwap``; it does not commit to pool assets, price, sale window, whitelist, release schedule, or ``address(this)``.

A valid signature can authorize arbitrary pool terms. In a re-org scenario, an old ``swap`` transaction may be replayed after the creator recreates the pool with different terms at the same index.

contracts/BounceFixedSwap.sol:L122-L138

```
function _createV2(
    uint256 id,
    CreateReq memory poolReq,
    ReleaseType releaseType,
    ReleaseData[] memory releaseData,
    uint256 expireAt,
    bytes memory signature
) private returns (uint256) {
    require(!ids[id], "id already exists");
    ids[id] = true;
    // make sure that msg.sender owns this signature signed by the signer.
    checkCreator(keccak256(abi.encode(id, PoolType.FixedSwap)), expireAt, signature);
    uint256 index = _create(poolReq);
    // set release data.
    setReleaseData(index, poolReq.claimAt, releaseType, releaseData);
    return index;
}
```

Recommendation

Use EIP-712 and sign ``address(this)``, all ``CreateReq`` fields, ``releaseType``, and ``releaseData``. Consider requiring ``swap`` callers to provide an expected pool-configuration hash.

Status

This issue has been acknowledged by the team.

5. Small Swaps May Pay Tokens but Receive Zero Output

Severity: Low

Category: Business Logic

Target:

- contracts/BounceFixedSwap.sol

Description

The function `\_swap` calculates output with integer division: $\text{amount0} = \text{amount1} * \text{amountTotal0} / \text{amountTotal1}$

The value is rounded down, but the function does not reject $\text{amount0} == 0$.

For sufficiently small payments, a user can transfer a positive amount of `token1` but receive zero `token0`.

contracts/BounceFixedSwap.sol:L221-L225

```
function _swap(uint256 index, uint256 amount1) private {
    uint256 amount0 = amount1.mul(pool.amountTotal0).div(pool.amountTotal1);
    uint256 _amount0 = pool.amountTotal0.sub(amountSwap0P[index]);
    if (_amount0 > amount0) {
        _amount0 = amount0;
    }
}
```

Recommendation

Require a nonzero output amount and preferably accept a user-provided `minAmount0` parameter.

Status

This issue has been acknowledged by the team.

2.3 Informational Findings

6. Critical Configuration Changes Lack Events

Severity: Informational

Category: Code Quality

Target:

- contracts/BounceBase.sol

Description

Changes to `txFeeRatio`, `stakeContract`, and `signer` update storage without emitting events.

Indexers, monitoring systems, and users cannot reliably reconstruct configuration changes from logs, reducing transparency and operational observability.

contracts/BounceBase.sol:L199-L202

```
function _setTxFeeRatio(uint256 _txFeeRatio) private {  
    require(_txFeeRatio <= TX_FEE_DENOMINATOR, "invalid txFeeRatio");  
    txFeeRatio = _txFeeRatio;  
}
```

Recommendation

Emit events for each configuration update, including both the previous and new values.

Status

This issue has been acknowledged by the team.

7. Unreachable Legacy Claim Branch

Severity: Informational

Category: Redundancy

Target:

- contracts/BounceFixedSwap.sol

Description

The `userClaim` fallback branch requires `claimAt != 0` and an empty `releaseDataList`. However, current pool creation requires non-instant pools with nonzero `claimAt` to have release data, while instant pools use `claimAt == 0`.

The branch is unreachable through the current creation flow, increasing maintenance burden and making the claim logic harder to reason about.

contracts/BounceFixedSwap.sol:L339-L347

```
function userClaim(uint256 index) external nonReentrant isPoolExist(index)
isClaimReady(index) {
    if (releaseDataList[index].length > 0) {
        ...
    } else {
        if (myAmountSwapped0[msg.sender][index] > 0) {
            // send token0 to msg.sender
            IERC20Upgradeable(pool.token0).safeTransfer(msg.sender,
myAmountSwapped0[msg.sender][index]);
        }
        myClaimed[msg.sender][index] = true;

        emit UserClaimed(index, msg.sender, myAmountSwapped0[msg.sender][index]);
    }
}
```

Recommendation

Remove the branch if legacy pools are not supported. If it is retained for upgrade compatibility, document its purpose and the expected legacy storage state.

Status

This issue has been acknowledged by the team.

Appendix

Appendix 1 - Files in Scope

This audit covered the following files:

File	SHA-1 hash
BounceBase.sol	bce6ed0d6abc9cde7a694effe2b6dfdbd973e795
BounceFixedSwap.sol	0a902b4a38b70be7692405d27ae4e80238eeead1